Assignment: Create REST API Using FastAPI

1. Aim

To design, develop, test, and document a REST API using FastAPI for serving predictions from a trained machine learning model.

2. Problem Statement

An ML model developed inside a Jupyter Notebook cannot be used directly by a web application, mobile application, or another software system. The trained model must be exposed through a standard communication interface.

In this assignment, students will train an Iris flower classification model and create a REST API using FastAPI. The API will accept flower measurements in JSON format and return the predicted flower species with a confidence score.

The API must provide:

- Root endpoint
- Health-check endpoint
- Model-information endpoint
- Single-prediction endpoint
- Input validation
- Error handling
- Automatic API documentation
- Unit testing

3. Learning Outcomes

After completing this assignment, students will be able to:

1. Explain REST API concepts and HTTP methods.
2. Create API endpoints using FastAPI.
3. Define and validate request data using Pydantic.
4. Load a trained ML model into an API application.
5. accept input in JSON format.
6. Return model predictions as JSON responses.
7. Handle invalid requests and prediction failures.
8. Test API endpoints using Swagger UI and `pytest`.
9. Explain how the API can be containerized and deployed on AWS.

4. Prerequisite Knowledge

Students should understand:

- Basic Python programming
- Python functions and classes
- Fundamentals of machine learning
- Model training using scikit-learn
- JSON data format
- Basic HTTP concepts
- Command-line operations

5. Theoretical Background

5.1 What is an API?

An Application Programming Interface, or API, is a mechanism that allows two software applications to communicate.

For example:

- A mobile application sends student data to an API.
- The API passes the data to an ML model.
- The model generates a prediction.
- The API returns the prediction to the mobile application.

5.2 What is a REST API?

REST stands for **Representational State Transfer**. A REST API exposes resources or operations through URLs called endpoints and uses standard HTTP methods.

HTTP method	Purpose	Example
`GET`	Retrieve information	Check API health
`POST`	Submit data	Generate a prediction
`PUT`	Replace data	Replace model configuration
`PATCH`	Partially update data	Update prediction threshold
`DELETE`	Delete data	Remove a model version

A prediction endpoint generally uses `POST` because the client sends feature values in the request body.

5.3 Components of a REST request

```
POST /predict HTTP/1.1
Content-Type: application/json
{
  "sepal_length": 5.1,
  "sepal_width": 3.5,
  "petal_length": 1.4,
  "petal_width": 0.2
}
```

Component	Description
Method	Operation to be performed, such as `GET` or `POST`

Endpoint	URL representing an API operation
Header	Metadata about the request
Request body	Input data sent to the API
Status code	Result of processing the request
Response body	Output returned by the API

5.4 What is FastAPI?

FastAPI is a Python framework used to develop web APIs. It supports:

- Python type hints
- Request and response validation
- JSON serialization
- Asynchronous programming
- OpenAPI schema generation
- Interactive Swagger documentation

FastAPI automatically creates Swagger UI at `/docs`, ReDoc at `/redoc`, and an OpenAPI schema at `/openapi.json`. FastAPI first steps

FastAPI uses Pydantic models to read JSON request bodies, convert data types, validate values, and generate documentation. FastAPI request bodies

6. Use-Case Architecture

Processing sequence

1. The client sends flower measurements.
2. FastAPI receives the JSON request.
3. Pydantic validates the input.
4. The validated features are passed to the model.
5. The model predicts the flower species.
6. FastAPI returns the prediction and confidence score.

7. Software Requirements

Students must install:

- Python 3.10 or later
- FastAPI
- scikit-learn
- NumPy
- Joblib
- Pytest
- HTTPX
- VS Code, PyCharm, or another Python IDE

8. Project Structure

Create the following folder structure:

```
fastapi-ml-assignment/
├── artifacts/
│   └── iris_model.joblib
├── tests/
│   └── test_api.py
├── main.py
├── train_model.py
├── requirements.txt
└── README.md
```

9. Implementation Procedure

Step 1: Create a virtual environment

Windows:

```
python -m venv venv
venv\Scripts\activate
```

Linux or macOS:

```
python3 -m venv venv
source venv/bin/activate
```

Step 2: Create `requirements.txt`

```
fastapi[standard]
scikit-learn
numpy
joblib
pytest
httpx
```

Install the dependencies:

```
pip install -r requirements.txt
```

10. Train and Save the ML Model

Create `train_model.py`:

```
from pathlib import Path

import joblib
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
```

```
# Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Divide the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# Train the classification model
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)

# Evaluate the model
predictions = model.predict(X_test)
test_accuracy = accuracy_score(y_test, predictions)

# Store the model and related metadata
model_bundle = {
    "model": model,
    "target_names": iris.target_names.tolist(),
    "feature_names": iris.feature_names,
    "test_accuracy": float(test_accuracy),
    "model_version": "1.0.0"
}

# Create the artifacts directory
artifact_directory = Path("artifacts")
artifact_directory.mkdir(exist_ok=True)

# Save the model
model_path = artifact_directory / "iris_model.joblib"
joblib.dump(model_bundle, model_path)

print(f"Model saved at: {model_path}")
print(f"Test accuracy: {test_accuracy:.4f}")
```

Run the training program:

```
python train_model.py
```

Expected output:

```
Model saved at: artifacts/iris_model.joblib
Test accuracy: 0.9000
```

The exact accuracy may vary if students modify the model or training parameters.

11. Develop the FastAPI Application

Create `main.py`:

```
from contextlib import asynccontextmanager
from pathlib import Path

import joblib
import numpy as np
from fastapi import FastAPI, HTTPException, Request
from pydantic import BaseModel, Field


MODEL_PATH = Path("artifacts/iris_model.joblib")


@asynccontextmanager
async def lifespan(app: FastAPI):
    """
    Load the model once when the application starts.
    """

    if not MODEL_PATH.exists():
        raise RuntimeError(
            "Model file not found. Run train_model.py first."
        )

    app.state.model_bundle = joblib.load(MODEL_PATH)

    yield

    # Release the model during application shutdown
    app.state.model_bundle = None


app = FastAPI(
    title="Iris Classification API",
    description="REST API for Iris flower classification",
    version="1.0.0",
    lifespan=lifespan
)


class PredictionRequest(BaseModel):
    sepal_length: float = Field(
        gt=0,
        le=20,
        description="Sepal length in centimetres"
    )

    sepal_width: float = Field(
        gt=0,
        le=20,
        description="Sepal width in centimetres"
    )

    petal_length: float = Field(
        gt=0,
        le=20,
        description="Petal length in centimetres"
```

```
    )

    petal_width: float = Field(
        gt=0,
        le=20,
        description="Petal width in centimetres"
    )


class PredictionResponse(BaseModel):
    predicted_class: int
    predicted_species: str
    confidence: float
    probabilities: dict[str, float]
    model_version: str


@app.get("/")
def root():
    """
    Display basic API information.
    """

    return {
        "message": "Welcome to the Iris Classification API",
        "documentation": "/docs",
        "health_check": "/health"
    }


@app.get("/health")
def health_check(request: Request):
    """
    Check whether the API and model are available.
    """

    model_loaded = (
        request.app.state.model_bundle is not None
    )

    return {
        "status": "healthy" if model_loaded else "unhealthy",
        "model_loaded": model_loaded
    }


@app.get("/model-info")
def model_information(request: Request):
    """
    Return model metadata.
    """

    bundle = request.app.state.model_bundle

    return {
        "model_type": type(bundle["model"]).__name__,
        "model_version": bundle["model_version"],
        "feature_names": bundle["feature_names"],
        "target_names": bundle["target_names"],
        "test_accuracy": bundle["test_accuracy"]
    }
```

```
@app.post(
    "/predict",
    response_model=PredictionResponse
)
def predict(
    data: PredictionRequest,
    request: Request
):
    """
    Generate an Iris flower prediction.
    """

    bundle = request.app.state.model_bundle
    model = bundle["model"]
    target_names = bundle["target_names"]

    features = np.array(
        [[
            data.sepal_length,
            data.sepal_width,
            data.petal_length,
            data.petal_width
        ]]
    )

    try:
        predicted_class = int(model.predict(features)[0])
        class_probabilities = model.predict_proba(features)[0]

    except Exception as error:
        raise HTTPException(
            status_code=500,
            detail="The model could not generate a prediction."
        ) from error

    probability_dictionary = {
        species: round(float(probability), 4)
        for species, probability in zip(
            target_names,
            class_probabilities
        )
    }

    confidence = float(
        class_probabilities[predicted_class]
    )

    return {
        "predicted_class": predicted_class,
        "predicted_species": target_names[predicted_class],
        "confidence": round(confidence, 4),
        "probabilities": probability_dictionary,
        "model_version": bundle["model_version"]
    }
```

12. Explanation of Important Code Elements

FastAPI application

```
app = FastAPI()
```

This creates the FastAPI application.

Request model

```
class PredictionRequest(BaseModel):
```

This class specifies the input fields expected by the API. Pydantic validates every submitted value.

For example:

```
sepal_length: float = Field(gt=0, le=20)
```

The input must:

- Be numeric
- Be greater than zero
- Be less than or equal to 20

Path operation decorator

```
@app.post("/predict")
```

This instructs FastAPI to execute the associated function when the server receives a `POST` request at `/predict`.

Response model

```
response_model=PredictionResponse
```

The response model defines the expected output structure. FastAPI can use it to validate, document, serialize, and filter returned data. FastAPI response models

Application state

```
app.state.model_bundle
```

The trained model is loaded once during application startup and stored in application state. It should not be loaded again for every prediction request.

Error handling

```
raise HTTPException(
    status_code=500,
```

```
    detail="The model could not generate a prediction."
)
```

`HTTPException` returns a controlled HTTP error instead of exposing internal program details.

13. Run the FastAPI Application

Run:

```
fastapi dev main.py
```

Alternatively:

```
uvicorn main:app --reload
```

Expected terminal output:

```
Application startup complete.
Uvicorn running on http://127.0.0.1:8000
```

Open the following URLs:

URL	Purpose
`http://127.0.0.1:8000/`	Root endpoint
`http://127.0.0.1:8000/health`	Health check
`http://127.0.0.1:8000/model-info`	Model information
`http://127.0.0.1:8000/docs`	Swagger UI
`http://127.0.0.1:8000/redoc`	ReDoc
`http://127.0.0.1:8000/openapi.json`	OpenAPI schema

14. Test the API Using Swagger UI

1. Open `http://127.0.0.1:8000/docs`.
2. Select `POST /predict`.
3. Click **Try it out**.
4. Enter the JSON input.
5. Click **Execute**.
6. Observe the request URL, response body, and status code.

Valid request

```
{
  "sepal_length": 5.1,
  "sepal_width": 3.5,
  "petal_length": 1.4,
  "petal_width": 0.2
}
```

Expected response

```
{
  "predicted_class": 0,
  "predicted_species": "setosa",
  "confidence": 1.0,
  "probabilities": {
    "setosa": 1.0,
    "versicolor": 0.0,
    "virginica": 0.0
  },
  "model_version": "1.0.0"
}
```

Probability values may vary slightly according to the trained model.

15. Test Invalid Input

Submit:

```
{
  "sepal_length": -5,
  "sepal_width": 3.5,
  "petal_length": 1.4,
  "petal_width": 0.2
}
```

FastAPI should reject the request because `sepal_length` is negative.

Expected status:

```
422 Unprocessable Content
```

The exact validation-response wording can differ slightly by installed FastAPI and Pydantic versions.

16. Test Using cURL

Health endpoint

```
curl http://127.0.0.1:8000/health
```

Prediction endpoint

```
curl -X POST "http://127.0.0.1:8000/predict" \
-H "Content-Type: application/json" \
-d '{
  "sepal_length": 5.1,
  "sepal_width": 3.5,
  "petal_length": 1.4,
  "petal_width": 0.2
}'
```

17. Automated API Testing

Create `tests/test_api.py`:

```
import pytest
from fastapi.testclient import TestClient

from main import app


@pytest.fixture
def client():
    with TestClient(app) as test_client:
        yield test_client


def test_root_endpoint(client):
    response = client.get("/")

    assert response.status_code == 200
    assert "message" in response.json()


def test_health_endpoint(client):
    response = client.get("/health")
    result = response.json()

    assert response.status_code == 200
    assert result["status"] == "healthy"
    assert result["model_loaded"] is True


def test_valid_prediction(client):
    request_data = {
        "sepal_length": 5.1,
        "sepal_width": 3.5,
        "petal_length": 1.4,
        "petal_width": 0.2
    }

    response = client.post(
        "/predict",
        json=request_data
    )

    result = response.json()

    assert response.status_code == 200
    assert result["predicted_species"] in [
        "setosa",
        "versicolor",
        "virginica"
    ]
    assert 0 <= result["confidence"] <= 1
    assert "probabilities" in result


def test_negative_feature_value(client):
    request_data = {
        "sepal_length": -5.1,
        "sepal_width": 3.5,
        "petal_length": 1.4,
        "petal_width": 0.2
```

```
    }

    response = client.post(
        "/predict",
        json=request_data
    )

    assert response.status_code == 422


def test_missing_feature(client):
    request_data = {
        "sepal_length": 5.1,
        "sepal_width": 3.5,
        "petal_length": 1.4
    }

    response = client.post(
        "/predict",
        json=request_data
    )

    assert response.status_code == 422
```

Run the tests:

```
pytest -v
```

Expected output:

```
test_root_endpoint PASSED
test_health_endpoint PASSED
test_valid_prediction PASSED
test_negative_feature_value PASSED
test_missing_feature PASSED
```

FastAPI's `TestClient` can test an application without creating an actual network connection. [FastAPI TestClient]

18. Required API Test Cases

Test	Input condition	Expected status
Root endpoint	`GET /`	`200`
Health check	Model loaded	`200`
Model information	`GET /model-info`	`200`
Valid prediction	All values valid	`200`
Negative value	One feature below zero	`422`
Missing feature	`petal_width` omitted	`422`
Incorrect data type	Text instead of a number	`422`
Unknown endpoint	`GET /unknown`	`404`

19. HTTP Status Codes to Observe

Status code	Meaning
`200`	Request successfully processed
`404`	Endpoint not found
`422`	Request validation failed
`500`	Internal prediction failure
`503`	Model or service unavailable

20. Expected Learning Result

After executing the assignment, students should demonstrate that:

- The trained model is saved as an artifact.
- FastAPI loads the model during startup.
- All required endpoints are accessible.
- Valid JSON produces a prediction.
- Invalid data is rejected automatically.
- Swagger documentation is generated.
- Automated tests pass successfully.

21. Student Tasks

Students must:

1. Create the required project structure.
2. Train and save an Iris classification model.
3. Implement all four API endpoints.
4. Define request and response schemas.
5. add appropriate validation rules.
6. Test the endpoints using Swagger UI.
7. Execute at least five automated test cases.
8. Document the request and response formats.
9. Prepare a `README.md` containing execution instructions.
10. Demonstrate the working application.